rbus

локальная высокопроизводительная шина сообщений для автомобильных и робототехнических систем

Проблема

Нужно передавать данные между процессами в рамках одного устройства

- Универсальные робототехнические фреймворки (по типу ROS 2) часто рассчитаны на широкий класс систем
- В них есть сетевые сценарии, разные режимы доставки (QoS-ы) и куча runtime-настроек
- Есть сценарии, где такая универсальность не требуется - например, когда процессы живут физически в одном месте

Отсюда возникла идея сделать локальный транспорт, который и работает шустро, и не ест много ресурсов CPU

Цель проекта

Сделать такой транспорт

Поскольку основным паттерном взаимодействия в такого рода системах является publish/subscribe, то основное внимание в работе будет уделено именно ему

Что требуется:

- **Производительность** — активное использование shared memory, loaned payload без копирования, блокирующий и неблокирующий пути передачи с минимальным количеством syscall-ов
- **Простая модель и локальный scope** — простой и понятный API, одно устройство, один daemon, без попытки сделать универсальный middleware
- **Надёжность** — освобождение ресурсов при падении/штатной остановки процессов, restart daemon-а не должен ломать уже настроенный путь для данных
- **Простая и информативная диагностика** — cli-утилиты, stack trace-ы и т. д.

Границы проекта

Что не входит в текущий этап

- сетевой транспорт
- сложные политики доставки
- async/callback API
- serde-схемы
- C/Python bindings
- автоматический запуск daemon-а

Архитектура

Daemon строит связи, payload идёт напрямую

Control plane

Publisher ↔ **rbusd** ↔ **Subscriber**

- REGISTER / ATTACH / INSPECT идут через rbusd по Unix SOCK_SEQPACKET
- FD передаются через SCM_RIGHTS
- daemon владеет registry и lifecycle

Data plane

Publisher → **memfd MAP_SHARED** → **Subscriber**

- daemon не копирует и не маршрутизирует payload
- после attach hot path работает через shared memory

Flow

Publisher path

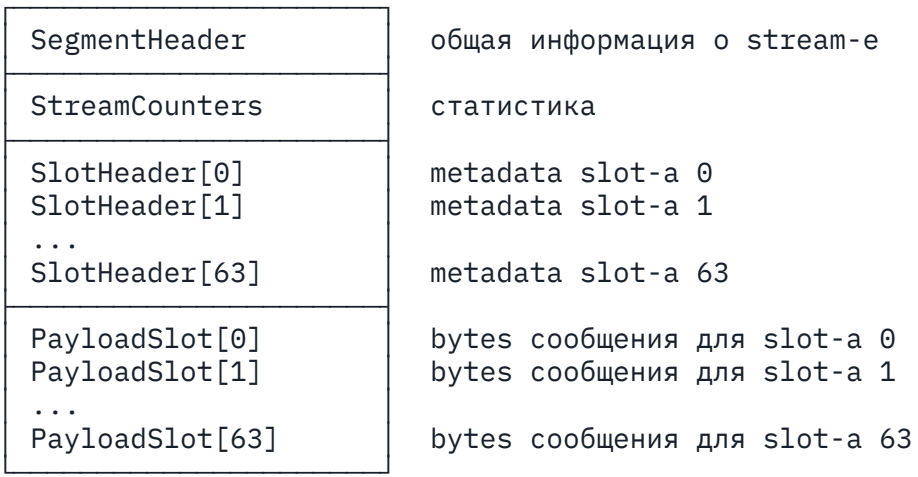
Bus::connect()
↓
bus.publisher(topic, TopicConfig::new(max_size, depth))
↓
rbus отправляет REGISTER_PUBLISHER в rbusd
↓
rbusd создаёт memfd stream segment
↓
rbusd делает ftruncate до нужного размера
↓
rbusd инициализирует header/ layout
↓
rbusd передаёт FD publisher-у через SCM_RIGHTS
↓
publisher mmap-ит segment

Subscriber path

bus.subscriber(topic)
↓
rbus отправляет REGISTER_SUBSCRIBER
↓
rbusd выдаёт subscriber_id и topic wait FD
↓
subscriber вызывает ATTACH_STREAMS
↓
rbusd передаёт FD всех stream-ов этого topic-а
↓
subscriber mmap-ит эти stream segment-ы

Ring в shared memory

Ring — фиксированный массив slot-ов, который используется циклически



Publisher пишет payload не «просто в stream», а в конкретный payload slot внутри stream-а:

```
сообщение #0  -> PayloadSlot[0]
сообщение #1  -> PayloadSlot[1]
сообщение #2  -> PayloadSlot[2]
...
сообщение #63 -> PayloadSlot[63]
сообщение #64 -> PayloadSlot[0] снова
сообщение #65 -> PayloadSlot[1] снова
```

Поэтому это называется ring: после последней ячейки возвращаемся в начало.

Гарантии hot path

Главные обещания data path

0

heap allocations: loan + publish

0

syscalls/message: non-blocking publish/try_recv

0

payload copies в loaned path

Как проверяется

- custom allocator tests
- marker-based strace smoke
- Loom-модели atomic protocol
- layout tests на cache-line counters

Технологии

Минимальный Linux/Rust стек

- **Rust 2024** — workspace package 0.1.0, warnings=deny, dead_code=deny
- **Linux IPC** — memfd_create, ftruncate, mmap MAP_SHARED, Unix SOCK_SEQPACKET, SCM_RIGHTS
- **Синхронизация** — futex для blocking recv, acquire/release atomics для slot protocol
- **Benchmarking** — perf stat, strace, CSV/JSON reports, regression gates

Методика измерений

Окружение и режимы нагрузки

- Ubuntu 26.04 LTS, Linux 7.0.0-27-generic
- Intel Core i7-11700F: 8C/16T
- governor: powersave, intel_pstate active
- cargo build –profile bench
- rbusd и rbus-bench из target/release

Режимы

- saturated / busy-poll: 4 publisher-а, 8 subscriber-ов
- payload: 64 B, 4 KiB, 64 KiB, 1 MiB
- paced / 30 Hz: controlled-load latency
- low-CPU recv: 5 секунд, 150 сообщений, 1P1S

Что сравниваем

Unix socket baseline против rbus SHM data path

- **Одинаковый scope** — локальный IPC на одной Linux-машине
- **Unix socket** — payload проходит через kernel socket path и socket buffer
- **rbus** — payload лежит в shared-memory ring, loaned path не копирует payload
- **Что проверяем** — что даёт переход от socket baseline к SHM data path

Результаты: throughput

Saturated busy-poll, 4P8S, payload 64 B



5.7x

loaned-fill vs unix socket

0

syscalls/message в rbus busy-poll data path

0

allocations/message в rbus busy-poll hot path

Результаты: большие payload

Saturated busy-poll, effective delivery GiB/s

Payload	loaned	publish	unix	x
4 KiB	53.8	59.1	8.2	6.5x
64 KiB	120.2	92.1	21.8	5.5x
1 MiB	146.1	154.1	15.6	9.4x

metadata-only

1 MiB: 661.2 GiB/s logical/effective

Это commit/control overhead без записи payload в measured window

Результаты: задержка под нагрузкой

Paced 30 Hz, 1P1S, busy-poll, p99 transport latency

Payload	loaned	publish	unix	x
64 B	1.2 us	2.4 us	126.7 us	105x
4 KiB	8.0 us	9.2 us	151.7 us	19x
64 KiB	23.5 us	23.8 us	276.1 us	12x
1 MiB	23.2 us	110.2 us	1615.5 us	70x

- Этот режим показывает нижнюю границу latency
- Он использует busy-poll и почти целое ядро на worker path
- Metadata-only не используется как основной latency-result

Результаты: низкий CPU в recv

Paced 30 Hz, 1P1S, wait-mode recv

64 B

- rbus loaned-fill: p99 81 us, 0.004 cores
- unix-socket: p99 103 us, 0.001 cores
- вывод: rbus быстрее по latency, Unix дешевле по CPU

1 MiB

- rbus loaned-fill: p99 231 us, 0.014 cores
- unix-socket: p99 1369 us, 0.019 cores
- вывод: rbus ниже по latency при сопоставимом CPU

Результаты: корректность

Benchmark-и не должны скрывать ошибки доставки

0

warning_count в выбранных строках

0

skipped_messages

0

publish deadline misses

Тесты

- cross-process data plane tests
- examples suite
- subscriber crash с удержанным MessageRef
- publisher crash
- rbusd restart

Сильные стороны и ограничения

Что получилось и где границы

Сильные стороны

- payload не проходит через daemon
- zero-copy loaned path
- высокий throughput на локальной машине
- microsecond-level paced latency в busy-poll
- низкий CPU recv path через futex

Ограничения

- Linux-specific IPC primitives
- API пока Rust-only
- нет сетевого транспорта
- один stream ограничен 63 subscriber reader bits
- low-CPU small payload не всегда дешевле Unix socket по CPU

Выводы

rbus закрывает локальный IPC-срез проекта

- **Цель достигнута** — реализован локальный message bus, где setup делает daemon, а payload идёт через shared memory
- **Ниша понятна** — fast local transport для процессов на одной машине, не замена универсального middleware
- **Результат измерен** — есть throughput, latency, perf/strace данные и regression tests

Следующие шаги

- adapter для внешнего robotics stack
- стабилизация API
- C++ binding для robotics-кода
- реальные sensor payloads
- долгие soak-тесты